

An End-to-End MLOps Pipeline for Urban Sound Classification

Risan Raja
Indian Institute of Technology Madras
Chennai, India

Abstract—This report describes a production-style machine learning operations (MLOps) system for ten-class urban sound event classification on UrbanSound8K. A host-side Apache Airflow DAG with LocalExecutor downloads parquet clips from Hugging Face Hub, cleans and augments waveforms, extracts tabular and log-mel features with PySpark in local mode, and trains four models under Optuna and MLflow: random forest, XGBoost, LightGBM, and a ResNet-18 convolutional network. The winning artifact is a ResNet-18 with holdout F1-macro 0.862 and official 10-fold CV F1-macro 0.760 ± 0.045 . Dataset intermediates and trained weights are versioned on Hugging Face Hub. A FastAPI service inside Docker Compose exposes /health, /metrics, and /predict; Prometheus and Grafana scrape request, latency, class, confidence, and drift series, and each prediction is written as structured JSON. GitHub Actions lint, test, and build the API image. The project was completed solo under an approved exception to the course team-size rule.

Index Terms—MLOps, audio classification, Airflow, PySpark, MLflow, FastAPI, UrbanSound8K

I. PROBLEM STATEMENT

Urban environments mix mechanical, social, and emergency sounds that a deployed classifier must name from a short waveform. The engineering problem is to move from a parquet dump to a served model with an audit trail: who trained what, on which fold split, with which hyperparameters, and how the live API behaves once the weights leave the laptop.

This project uses UrbanSound8K classification as the use case for that lifecycle [1]. A clip arrives as a .wav upload; the service returns a class label, a confidence, the serving model name, end-to-end latency, and the full ten-class probability vector. Behind that contract sit an orchestrated data path, two feature families, four compared models, experiment tracking, remote versioning, containerized serving, and request-level monitoring.

The course brief requires nine components: version control; an Airflow-orchestrated pipeline plus Spark, Kafka, or Beam; data processing; at least two compared models with MLflow; dataset and model versioning (DVC or equivalent); FastAPI or Flask plus Docker; logging and at least one of Prometheus, Kibana, drift detection, performance monitoring, or latency/prediction logging; CI/CD; and a README. The sections that follow map each requirement to a concrete choice and explain divergences from the usual Compose-everything sketch. Table I is the short form of that mapping.

TABLE I
BRIEF COMPONENT VERSUS WHAT SHIPPED

Brief item	This repo
Git + Hub	Feature branches, semantic commits, GitHub remote
Airflow + DE tool	Host LocalExecutor DAG; PySpark local UDFs
Processing	Decode, resample, peak-norm; train-fold augment
≥ 2 models + MLflow	RF, XGBoost, LightGBM, ResNet-18; SQLite tracker
Versioning	HF dataset prefixes + model repo (DVC documented)
Serve + Docker	FastAPI three-route API in Compose
Monitoring	Prometheus, Grafana, JSON logs, KS/PSI drift
CI/CD	GitHub Actions: ruff, pytest, image build/push
README	Architecture, setup, API, Docker, tree

II. DATASET

UrbanSound8K contains 8,732 labeled clips of at most four seconds, drawn from Freesound and grouped into ten classes: air conditioner, car horn, children playing, dog bark, drilling, engine idling, gun shot, jackhammer, siren, and street music [1]. Each row carries a fold integer in $\{1, \dots, 10\}$. Those folds are the official evaluation protocol: they keep recordings from the same source file out of both sides of a split, which a random shuffle does not.

The pipeline does not clone the original archive as a git submodule and does not use the Kaggle mirror. It fetches a parquet conversion of the same clips from the public Hub dataset risan-raja-iitm/urbansound8K (a clone of danavery/urbansound8K, CC-BY-NC-4.0). Audio lives as Hugging Face Audio structs inside sixteen parquet shards [2]. The verified columns are audio (bytes and path), slice_file_name, fsID, start, end, salience, fold, classID, and class. A downloader keyed on the Hub revision SHA writes a .manifest.json and validates row count (8,732), fold range, and class count before any later stage runs. A git submodule of the raw archive would have forced a multi-gigabyte clone plus git-lfs on every grader and CI runner; the Hub fetch keeps the git tree small and needs no Kaggle credentials.

Raw parquet is treated as an upstream cache, not as this project's versioned dataset. The versioning claim in Section IV applies to *this* pipeline's interim waveforms, processed features, and model artifacts. Class counts are uneven (gun shot and car horn are the smallest), which is why training-fold

augmentation exists and why macro-F1, not accuracy, is the selection metric.

III. SYSTEM ARCHITECTURE

Figure 1 is the runtime map: data preparation and training on the host, serving and monitoring in Docker Compose. Airflow and PySpark run on the host virtualenv. Docker Compose runs only the API, MLflow UI, Prometheus, and Grafana. The brief requires Airflow, not Airflow-in-Docker. Host LocalExecutor executes DAG callables in the same Python that trains ResNet-18, so Apple Silicon MPS is available and there is no second, CPU-only torch build inside a Linux container [3]. Celery, Redis, and Flower add failure modes without a second worker machine.

PySpark is the second data-engineering tool [4], [5]. It runs as `master=local[*]` (configured `local[2]` in `config/config.yaml`) inside a `PythonOperator`, not as a standalone cluster. Eight thousand short clips do not need a Spark cluster; the UDF surface stays the same if one is pointed at later.

MLflow uses a SQLite backend store under `./mlflow/`, bind-mounted into the Compose service so host training and the container UI share one file [6]. SQLite is a single-writer store: host training and the containerized `mlflow server` must not write the same file at once. The DAG therefore sets `MLFLOW_TRACKING_URI=http://localhost:5001` when the Compose tracker is up.

Versioning is two Hub repositories. The dataset repo already holding raw parquet also receives `interim/` and later `processed/` prefixes. A separate model-type repo (`risa-raja-iitm/urbansound8k-models`) stores the four trained directories plus `winner.json` and `winner/`. The brief allows “DVC or equivalent”; Hub dataset and model repos are the primary equivalent. DVC remains a documented fallback in the README, not the path the DAG pushes.

IV. DATA PIPELINE

The DAG `audio_classification` is manual (`schedule=None`). Figure 2 shows the task graph: a linear ingest/feature spine, a four-way fan-out for training, then fan-in for winner selection and Hub pushes. Each callable changes into the repository root and imports the corresponding `src` module only inside the task body, so the scheduler can parse the DAG without loading torch or Spark. `preprocessing.enabled` and `spark.enabled` in `config.yaml` can turn the heavy stages into no-ops so a demo rerun reuses `data/interim` and `data/processed` instead of spending an hour re-deriving wavs.

A. Cleaning and augmentation

Raw clips arrive with mixed sample rates, mono or stereo channels, and occasional empty or silent payloads. The preprocessor decodes bytes with `soundfile`, rejects empty arrays, collapses stereo by mean, resamples to 16 kHz with `librosa` [7], and peak-normalizes. Silent files (peak amplitude zero) raise;

they are not imputed. Corrupt or empty audio fails loudly rather than becoming a zero vector that later looks like a legitimate class.

Fold 10 is the held-out evaluation fold during preprocessing and is never augmented. Training folds receive one extra copy per clip from an `audiomentations` compose of Gaussian noise, pitch shift of ± 2 semitones, and time stretch in $[0.9, 1.1]$, each applied with probability 0.5. Seeds are hashed from a base seed, clip key, and copy index so reruns are deterministic. The resulting wavs plus a `metadata.parquet` (including an `is_augmented` flag) form `data/interim`.

B. PySpark feature extraction

The extractor starts a local `SparkSession` and maps each absolute wav path through a Python UDF [5]. Every clip is padded or truncated to 4s. Two feature families are written:

Tabular. Thirteen MFCCs and their first-order deltas, twelve chroma bins, spectral centroid, bandwidth, rolloff, and zero-crossing rate, each reduced to mean and standard deviation (84 scalars). These feed the tree models [8].

Log-mel. A 128×126 log-mel spectrogram ($n_{\text{fft}} = 1024$, hop 512) for the CNN, with per-dataset mean/variance stored for serving-time normalization.

The same `librosa` helpers live in `src/data_pipeline/audio_features.py` so the API can rebuild features from an uploaded wav without importing PySpark.

C. Fold protocol at train time

Training reads processed parquet and applies the configured holdout: folds 1–8 for fitting, fold 9 for Optuna validation, fold 10 for the reported eval metrics. When `training.cv.enabled` is true, the selected hyperparameters are re-evaluated under official 10-fold CV (train on nine folds, test on the held-out fold, rotate). Winner selection prefers `cv_f1_macro_mean` when present, otherwise holdout F1-macro.

V. MODEL DEVELOPMENT

Four models are trained, which is more than the brief’s minimum of two, and they sit on two representations rather than two hyperparameter grids of the same learner. Figure 3 is the per-model lifecycle shared by the tabular baselines and ResNet-18.

Random forest [9], XGBoost [10], and LightGBM [11] consume the 84-D tabular vector after a `StandardScaler` fit on the training folds only. Class-weight vectors derived from the Optuna training folds rebalance the minority gunshot and car-horn rows during the tree fits. ResNet-18 [12] consumes the normalized log-mel as a single-channel image; ImageNet residual weights are fine-tuned for ten classes with early stopping patience 5.

Search spaces live in the training modules, not in `config.yaml`, so the config file stays a list of paths and fold integers. Optuna runs 20 trials per model [13]. Random forest searches $n_{\text{estimators}} \in \{100, \dots, 400\}$, depth 4–24, leaf

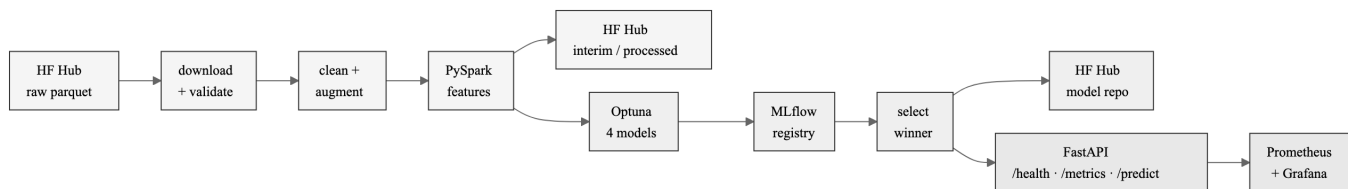


Fig. 1. System architecture. Hugging Face Hub stores raw and processed data plus model artifacts; Airflow/LocalExecutor and PySpark run on the host; FastAPI, Prometheus, and Grafana run in Compose.

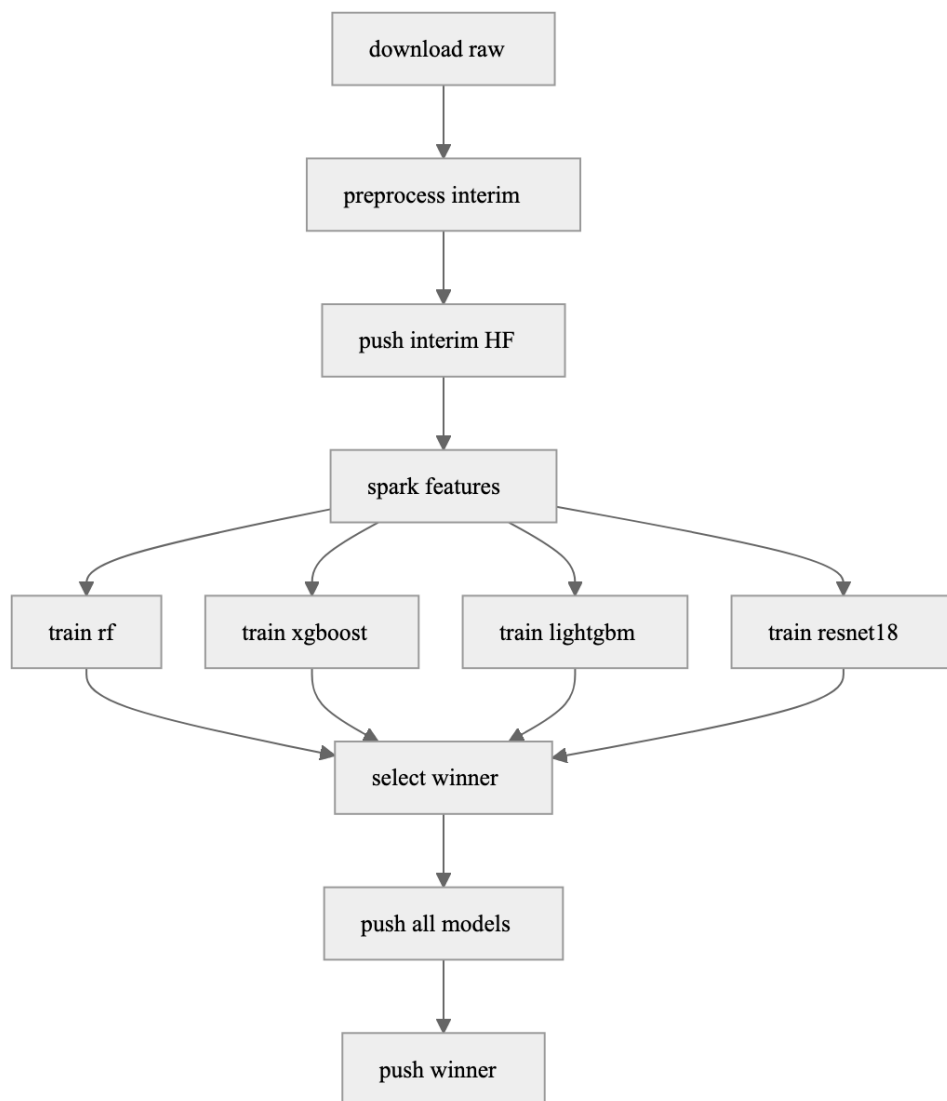


Fig. 2. Airflow DAG `audio_classification`. Sequential ingest through Spark feature extraction, fan-out to four parallel `train_*` tasks, fan-in at `select_winner`, then Hub pushes.

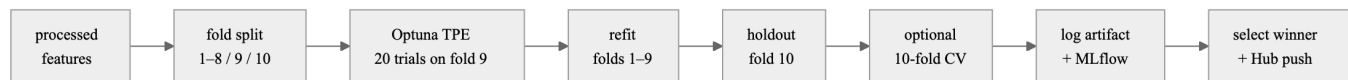


Fig. 3. Per-model development lifecycle. Optuna maximizes fold-9 macro-F1 over 20 trials; best hyperparameters refit folds 1–9, score fold 10, optionally run 10-fold CV, then log to MLflow and enter winner selection.

TABLE II
OPTUNA SEARCH SPACES (20 TRIALS EACH)

Model	Tuned knobs
RF	trees 100–400; depth 4–24; leaf; max_features
XGBoost	trees; depth 3–12; η ; subsample; λ
LightGBM	trees; depth; η ; leaves 16–128; min child
ResNet-18	$\eta \in [10^{-5}, 3 \times 10^{-4}]$; batch {16, 32}

size, and max_features. XGBoost and LightGBM search tree depth, log-scaled learning rate, column/row subsample, and L_2 regularization; both use histogram tree methods so four parallel Airflow tasks stay on CPU without GPU contention. The CNN trial space is smaller on purpose: log-uniform learning rate in $[10^{-5}, 3 \times 10^{-4}]$ and batch size {16, 32}. A twenty-trial search over residual architecture would have blown the wall clock; the capacity is already in the pretrained trunk.

After the best trial, the code refits on folds 1–9 (train plus the Optuna val fold) and scores fold 10. That refit is what models/<name>/ and the MLflow registry actually store. Nested MLflow runs keep each Optuna trial visible under the parent study so a viva can open one experiment and see why LightGBM beat random forest without opening four notebook dumps.

Table II lists the searched knobs. The spaces are wide enough that twenty trials sample rather than exhaust the grid.

Every trial logs parameters and the eval-fold metrics required by the brief: accuracy, macro precision, macro recall, macro F1, weighted F1, and one-vs-rest ROC-AUC, plus a confusion-matrix PNG. The fitted artifact, scaler or mel statistics, and label maps are written under models/<name>/ and registered in MLflow. After the four train_* tasks finish, select_winner copies the best directory to models/winner/, writes winner.json (model name, scores, MLflow run_id, dataset SHA), and tags the registry alias production.

OpenMP and BLAS thread caps are loaded from .env in src/models/runtime_env.py before sklearn, XGBoost, LightGBM, or torch import. Without that, four parallel Airflow train tasks over-subscribe the host and the CNN’s MPS session fights the tree libraries for cores.

VI. DEPLOYMENT

The serving surface is three endpoints on FastAPI [14], containerized from docker/Dockerfile.api (python:3.11-slim) and published on host port 8000.

/health returns {"status": "ok"} even when weights failed to load, so an orchestrator can distinguish “process up” from “model ready.” /metrics emits Prometheus text. /predict accepts a multipart .wav. Uploads over 8 MiB, empty bodies, and files that are neither .wav by suffix nor audio/wav (and close aliases) by MIME are rejected with 400 or 413. A missing winner returns 503. On success

the body is the Pydantic model: label, confidence, model_name, latency_ms, and probabilities.

Startup resolves artifacts in order: models/winner/, then models/<name>/ from winner.json, then a Hub snapshot of the model repo into a cache directory. A CNN directory must contain model.pt, mel_stats.json, and label_map.json; a tabular directory must contain model.joblib, scaler.joblib, and label_map.json. CNN and tabular families share predict_clip: decode, resample, pad or truncate, then either a torch forward pass or a scaler-plus-tree predict_proba. Compose bind-mounts config/ and models/ read-only and loads .env. DA5402W_CONFIG and DA5402W_MODELS_DIR override those mounts when the same code is run under uvicorn on the host. A helper, make demo-predict, posts three UrbanSound8K clips per class so Grafana is not an empty shell during a demo.

VII. MONITORING

The brief asks for logging plus one item from a five-option list. This stack implements four of those items: Prometheus scrape metrics, API latency histograms, per-prediction JSON logs, and a KS/PSI drift detector. Grafana is provisioned on top (not required by the brief) so a viva can show request rate, error ratio, p50/p95/p99 latency, predicted-class mix, mean confidence, class-mix PSI, confidence KS, and process CPU/memory without clicking through PromQL.

Counters and histograms are predict_requests_total, predict_errors_total, predict_class_total{class_name}, predict_latency_seconds, and predict_confidence [15]. Drift gauges are drift_psi_class, drift_ks_confidence, and drift_window_size. The API Overview dashboard is loaded from docker/grafana/provisioning/ on container boot. Each request, success or failure, is also written to stdout as one JSON object with timestamp, status, label, confidence, model name, latency, and filename.

Live drift compares a rolling window of 100 successful predictions to models/drift_reference.json. Class-mix PSI uses the fold-10 label prior (official UrbanSound8K counts when data/processed is absent). Confidence KS uses fold-10 quantile samples when those are present in the reference; otherwise the KS gauge stays unset. The CNN serving path does not extract the 84-D tabular vector, so feature PSI/KS is an offline CLI (python -m src.monitoring.drift_cli) against processed parquet, not a per-request cost.

VIII. CI/CD

.github/workflows/ci.yml runs on pull requests and pushes to main. Three jobs chain: ruff check, pytest -m "not integration", then a Docker build of Dockerfile.api. On main the image is pushed to GitHub Container Registry as

TABLE III
HOLDOUT (FOLD 10) AND 10-FOLD CV. WINNER SELECTED BY CV
F1-MACRO.

Model	Acc.	F1-mac.	AUC	CV F1
RF	0.720	0.721	0.956	—
XGBoost	0.771	0.776	0.961	0.689 ± 0.053
LightGBM	0.780	0.788	0.960	0.697 ± 0.054
ResNet-18	0.848	0.862	0.987	0.760 ± 0.045

ghcr.io/<owner>/da5402w-api with latest and git-SHA tags. PR builds verify the Dockerfile without publishing.

Integration tests that hit the Hub or a live Spark session are marked and excluded so CI stays deterministic on a GitHub-hosted runner. The workflow does not boot the full Compose stack. A multi-service Airflow-plus-API boot inside that runner is a flake source that does not improve the Dockerfile check the brief actually needs.

IX. REPRODUCIBILITY

The git tree is meant to be cloned without seven gigabytes of wavs. `uv sync` installs the lockfile; Java 17 is the only extra host package PySpark needs. `make compose` starts the API, MLflow on host port 5001 (macOS AirPlay already owns 5000), Prometheus, and Grafana. `make airflow` starts LocalExecutor with metadata in `gitignored`. `make pull-winner` snapshots `winner.json` and `winner/` from the model repo if those directories are absent.

A reader who only wants to score a clip never runs Spark. The serving container rebuilds features from the uploaded bytes with the shared librosa helpers, using the scaler or mel statistics that travelled with the winner. That is why feature code was lifted out of the Spark module: importing `pyspark` inside the slim API image would have pulled Java into a service that only needs `numpy` and `librosa`.

Configuration is one YAML file. Fold lists, trial count, Spark heap, Hub repo ids, and push flags are all there. Secrets stay in `.env` (Hugging Face token, Grafana admin password). Grafana defaults to `admin/admin` when those variables are unset, which is acceptable for a local demo and would not be acceptable on a shared course server.

X. RESULTS AND DISCUSSION

Table III reports fold-10 holdout metrics after Optuna and the subsequent official 10-fold CV mean and standard deviation of macro-F1. Numbers are taken from `models/*/metrics.json`.

ResNet-18 is the winner on both the holdout F1-macro (0.862) and CV F1-macro (0.760 ± 0.045). Among tabular models LightGBM leads holdout F1 (0.788) and CV F1 (0.697 ± 0.054); XGBoost is close on CV (0.689 ± 0.053). Random forest is a usable baseline (0.721 holdout F1) but trails both boosters; that run’s `metrics.json` has no CV block, so the table leaves the cell blank rather than inventing a number.

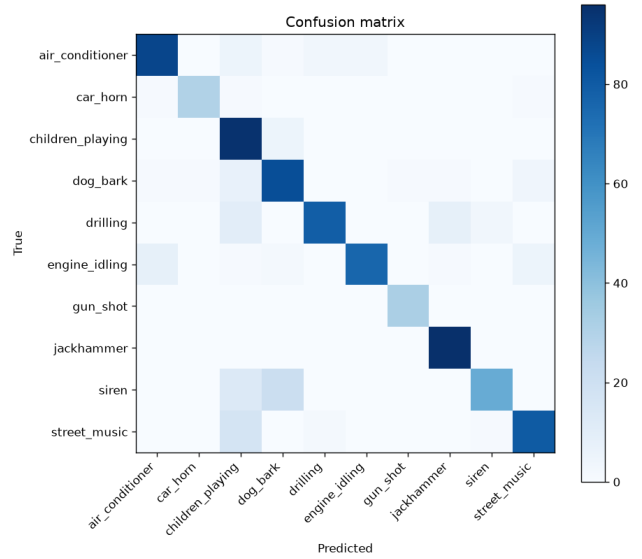


Fig. 4. ResNet-18 holdout confusion matrix (fold 10). Mechanical pairs (drill/jackhammer, engine/AC) and siren versus vocal street scenes remain the main errors.

The gap between holdout and CV is why CV exists in the DAG. Fold 10 happened to be a favorable eval slice; rotating the test fold drops macro-F1 by about 0.09–0.10 and exposes fold-to-fold variance (ResNet CV std 0.045). Fold-10-only reporting would overstate deployed quality. CV-only reporting would hide the Optuna objective (fold 9) and the published holdout (fold 10), both of which a reader can still recompute from `winner.json`.

Figure 4 is the ResNet-18 holdout confusion matrix. The diagonal is dark for air conditioner, children playing, dog bark, drilling, engine idling, jackhammer, and street music. Car horn and gun shot are the minority classes and therefore lighter on a raw-count heatmap even when recall is acceptable. The visible mistakes are the ones UrbanSound8K papers usually report: siren leaking into children playing and dog bark; drilling exchanging mass with jackhammer; engine idling with air conditioner; street music with children playing. Those pairs overlap in spectrum and in the original recordings’ background.

Figure 5 is LightGBM on the same fold. The error structure is the same family, with a thicker siren-to-children-playing off-diagonal. The CNN’s extra 0.08 holdout F1 comes from cutting the confusions that a bag of MFCC and chroma summaries cannot separate.

Lineage in `winner.json` records the Hub dataset id, processed-set timestamp, tabular SHA, and MLflow run `c0057386aa71433883bf00dab8044e9f`. A later re-train that does not match that SHA is a different artifact, even if the class name is still `resnet18`.

Reproducibility for a grader is a short list: `clone`, `uv sync`, `make pull-winner` or a local `models/winner`, `make compose`, and `curl` a sample wav. Full training is optional;

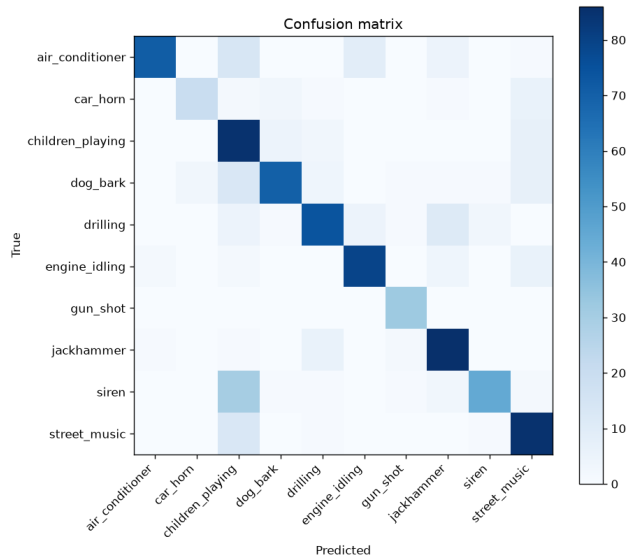


Fig. 5. LightGBM holdout confusion matrix. The same class pairs confuse the tabular model, with more siren→children_playing mass than ResNet-18.

the Hub model repo is the artifact store the brief asked for. The DAG remains the path that created those artifacts, and that is what the viva should walk.

XI. CHALLENGES

Putting Airflow in Compose looked like the default “production” sketch and is the wrong default on this machine. A Linux API image cannot see MPS; a Celery worker in Docker would train ResNet on CPU and turn a twenty-trial CNN search into an overnight job. Host LocalExecutor plus a thin Compose file (API, tracker, scrape stack) was the combination that actually finished.

PySpark on the host needed Java 17 and a small driver heap (1g, 512m result cap, two partitions, batch 512). The first instinct—a large Spark driver—wasted RAM without raising throughput on 4s wavs. The UDF already releases the GIL around librosa; parallelism comes from partitions, not from a 16g heap.

MLflow’s SQLite file cannot have two writers. Early in development a host mlflow ui and the Compose server locked each other. The rule is now mechanical: either point training at <http://localhost:5001> or stop Compose before writing `sqlite:///mlflow/mlflow.db`.

Four Optuna studies in parallel collide on OpenMP. Setting `OMP_NUM_THREADS` (and related knobs) in `.env` and importing them before the ML stack was the difference between a clean DAG run and a stall that looked like a Spark hang.

Finally, the versioning sentence in the brief is easy to misread as “put UrbanSound8K itself in DVC.” The raw Hub snapshot is someone else’s dataset. What this project versions is interim audio, processed parquet, and the four model directories—the artifacts a teammate would need to reproduce the winner without re-deriving features from scratch.

XII. FUTURE IMPROVEMENTS

Grafana should grow alert rules on error ratio, p95 latency, and PSI rather than remaining a demo dashboard.

CI can gain a Compose smoke that hits `/health` and `/predict` with the checked-in sample wav, gated so it does not run on every PR. DVC as a second remote (local filesystem) is still only documented; wiring `dvc.yaml` stages that mirror the DAG would make the “or equivalent” claim bidirectional.

On the model side, class-balanced sampling or a siren-specific augment would attack the off-diagonal that both figures share. The remaining errors are acoustic.

ACKNOWLEDGMENT

The course brief requires teams of two to four students. This submission is solo work under an approved exception.

REFERENCES

- [1] J. Salamon, C. Jacoby, and J. P. Bello, “A dataset and taxonomy for urban sound research,” in *Proc. 22nd ACM Int. Conf. Multimedia*. ACM, 2014, pp. 1041–1044.
- [2] Q. Lhoest, A. Villanova del Moral, Y. Jernite, A. Thakur, P. von Platen, S. Patil, J. Chaumond, M. Drame, J. Plu, L. Tunstall *et al.*, “Datasets: A community library for natural language processing,” arXiv:2109.02846, 2021.
- [3] Apache Software Foundation, “Apache Airflow,” <https://airflow.apache.org/>, 2024.
- [4] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Spark: Cluster computing with working sets,” in *Proc. 2nd USENIX Conf. Hot Topics in Cloud Computing*, 2010.
- [5] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, “Apache Spark: A unified engine for big data processing,” in *Commun. ACM*, vol. 59, no. 11, 2016, pp. 56–65.
- [6] M. Zaharia, A. Chen, A. Davidson, A. Ghodsi, S. A. Hong, A. Konwinski, S. Murching, T. Nykodym, P. Ogilvie, M. Parkhe, F. Xie, and C. Zumar, “Accelerating the machine learning lifecycle with MLflow,” in *IEEE Data Eng. Bull.*, vol. 41, no. 4, 2018, pp. 39–45.
- [7] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, “librosa: Audio and music signal analysis in Python,” *Proc. 14th Python in Science Conf.*, pp. 18–25, 2015.
- [8] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, “Scikit-learn: Machine learning in Python,” *J. Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [9] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [10] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*. ACM, 2016, pp. 785–794.
- [11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. IEEE Conf. Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [13] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A next-generation hyperparameter optimization framework,” in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*. ACM, 2019, pp. 2623–2631.
- [14] S. Ramírez, “FastAPI,” <https://fastapi.tiangolo.com/>, 2024.
- [15] B. Rabenstein and J. Volz, “Prometheus: A next-generation monitoring system,” in *SREcon15 Europe*. USENIX, 2015.